

Autonomous God-Level Master Prompt

Self-Improving, Model-Agnostic Personal AI Assistant

> **Purpose:** This is a master prompt you can paste into a strong reasoning model to make it design a complete architecture, roadmap, and implementation blueprint for a personal AI assistant that learns over time, remembers across sessions, generates reusable skills, supports multiple LLM providers, runs continuously on a cheap cloud server, and is accessible through CLI plus messaging apps.

How to use

1. Paste the full prompt from **Master Prompt** into Claude, GPT, Gemini, or another strong reasoning model.
 2. If needed, prepend your own constraints such as budget, preferred languages, or hosting provider.
 3. Ask the model to produce the output in phases: architecture first, then repo scaffold, then implementation tickets, then code.
 4. Use the model's output as a living technical spec, not a final truth; validate all security, privacy, and compliance choices before shipping.
-

Master Prompt

You are an elite AI systems architect, principal engineer, product strategist, platform designer, DevOps lead, and automation expert.

Your task is to design a complete, production-oriented, self-improving personal AI assistant platform that can learn and adapt over time.

You must act like a world-class technical founder and produce a deeply reasoned, highly structured, implementation-ready blueprint.

The assistant should feel like a personal operating layer for my digital life: always available, memory-rich, adaptive, and proactive.

You are not writing marketing copy. You are designing the real system.

CORE PRODUCT VISION

Design a personal AI assistant that:

- remembers past conversations and preferences across sessions,
- builds an evolving model of who I am,
- improves over time based on interactions and feedback,
- can convert successful complex task completions into reusable "skills",
- is not locked to a single LLM provider,
- can switch between OpenAI, Anthropic, OpenRouter, and similar providers via API integration,
- is accessible through both a polished command-line interface and messaging channels like Telegram and Discord,
- runs continuously on low-cost cloud infrastructure,
- supports scheduled and automated tasks like daily briefings, reminders, summaries, recurring jobs, and background repo management.

PRIMARY DESIGN GOAL

Create a comprehensive technical architecture and implementation roadmap for a self-improving, model-agnostic personal AI assistant.

- persistent memory,
- dynamic user modeling,
- autonomous skill generation,
- modular LLM routing,
- multi-interface accessibility,
- always-on low-cost deployment,
- robust scheduling and automation.

NON-NEGOTIABLE REQUIREMENTS

1) Persistent Memory Layer

The system must remember across sessions.

Design memory as a layered architecture, not a single notes table.

Include:

- episodic memory: key past interactions and events,
- semantic memory: stable facts about the user,
- procedural memory: learned workflows, preferences, and habits,
- working memory: current task context,
- reflective memory: summaries of patterns, lessons, and behavior changes over time.

The memory system must:

- continuously extract useful facts from conversations,
- avoid storing everything blindly,
- support retrieval by relevance, recency, confidence, and source,
- update beliefs when new evidence conflicts with old facts,
- allow manual review, editing, pinning, forgetting, and privacy controls,
- build a dynamic user profile that becomes more accurate with usage.

2) Autonomous Skill Generation Framework

The assistant must be able to create reusable skills from successful complex tasks.

A skill is a reusable unit that captures:

- task objective,
- input schema,
- preconditions,
- step sequence,
- tool usage,
- fallback logic,
- output schema,
- safety boundaries,
- evaluation criteria,
- metadata on when to reuse the skill.

The skill framework must:

- detect when a task is sufficiently repeatable,
- summarize successful execution traces into reusable skill definitions,
- allow skill versioning,
- support approval workflows before auto-activation,
- support skill replay, editing, disabling, and retirement,
- distinguish between user-authored skills, AI-synthesized skills, and imported skills,
- maintain an internal skill registry.

3) Model-Agnostic LLM Backend

The backend must be modular and provider-agnostic.

It should support plug-and-play integration with:

- OpenAI,
- Anthropic,
- OpenRouter,
- optionally Gemini, Groq, local models, and future providers.

The system must:

- switch models by configuration,
- route requests based on task type, budget, latency, and capability,
- separate provider adapters from core orchestration logic,
- normalize chat, tool-calling, streaming, embeddings, and structured output APIs,
- support fallback chains when a provider fails,
- support per-task model selection policies,
- support bring-your-own-key securely.

4) Multi-Channel Accessibility

The same assistant identity and memory must be accessible through multiple surfaces.

Design at minimum:

- a polished terminal-first CLI,
- a Telegram bot gateway,
- a Discord bot gateway,
- optional HTTP/web dashboard or admin panel for observability.

The interfaces must all connect to the same backend agent core and shared memory/skill systems.

Design interface-specific behavior without duplicating core logic.

5) Always-On Low-Cost Cloud Deployment

The system must run continuously on cheap infrastructure.

Optimize for a low monthly budget and realistic solo-developer operations.

Assume deployment to a small VPS or low-cost container host.

Consider options like:

- Hetzner,
- DigitalOcean,
- Railway,
- Fly.io,
- Render,
- VPS + Docker,
- optional managed services only where justified.

Design for:

- small footprint,
- low idle cost,
- resilience,
- backups,
- environment secrets,
- easy restart,
- observability,

- simple upgrades.

6) Scheduling and Automation

The assistant must support time-based and event-driven automation.

Include:

- cron-style scheduled jobs,
- one-off reminders,
- recurring digests,
- daily and weekly reports,
- background maintenance jobs,
- scheduled research tasks,
- notifications through CLI, Telegram, Discord, or email.

The scheduler must support:

- job persistence,
- retries,
- idempotency,
- status tracking,
- auditing,
- manual pause/resume,
- human approval for high-risk actions.

WHAT YOU MUST PRODUCE

Generate a complete answer with the following sections, in this exact order.
Be extremely detailed and concrete.

1. Executive Architecture Summary

Provide a concise but high-signal overview of the system, including:

- what the system is,
- how the major components work together,
- why the architecture is appropriate,
- core design principles,
- key trade-offs,
- recommended implementation philosophy.

2. Product Definition

Define:

- target user profile,
- user value proposition,
- core product capabilities,
- out-of-scope items for MVP,
- long-term product evolution,
- what "self-improving" should mean in practice,
- what must remain user-controlled versus autonomous.

3. High-Level System Architecture

Provide a high-level architecture diagram in text form.

Include at minimum these logical components:

- interface layer,
- API gateway,
- auth and identity layer,
- conversation/session manager,
- orchestration engine,
- LLM provider abstraction layer,
- tool execution layer,
- memory subsystem,
- user profile engine,
- skill generation subsystem,
- scheduler/automation engine,
- persistence layer,
- observability layer,
- admin/review layer.

Show request flow between:

- CLI to backend,
- Telegram/Discord to backend,
- task execution to memory capture,
- skill synthesis after successful tasks,
- scheduled jobs to outputs.

4. Core Subsystem Design

Design each subsystem in depth.

For every subsystem, include:

- purpose,
- responsibilities,
- boundaries,
- data flow,
- interfaces,
- failure modes,

- extension points.

Mandatory subsystems:

- conversation engine,
- memory engine,
- profile engine,
- retrieval and ranking engine,
- skill synthesis engine,
- skill runtime/registry,
- model router,
- provider adapters,
- tool/plugin framework,
- scheduling engine,
- notification gateway,
- interface adapters (CLI, Telegram, Discord),
- policy/safety layer,
- logging/metrics/tracing.

5. Memory Architecture

Design a sophisticated persistent memory system.

Include:

- memory taxonomy,
- storage strategy,
- extraction pipeline,
- consolidation pipeline,
- deduplication logic,
- memory conflict resolution,
- retrieval scoring formula,
- decay/aging policy,
- confidence model,
- user profile materialization,
- editable memory review workflow,
- privacy and forgetfulness controls.

Specify exactly which data belongs in:

- relational DB,
- vector DB,
- document store,
- cache.

Define the lifecycle of a memory item from:

conversation -> candidate fact -> scored memory -> retrieval -> revision -> archival/deletion.

6. User Modeling and Personalization Engine

Design a dynamic user model that updates over time.

Include dimensions like:

- identity facts,
- preferences,
- goals,
- recurring tasks,
- communication style,
- technical depth,
- schedule patterns,
- trust boundaries,
- sensitive domains,
- feedback history.

Explain how the model is:

- inferred,
- validated,
- updated,
- surfaced to the assistant,
- corrected by the user,
- protected from hallucinated assumptions.

7. Autonomous Skill Generation Framework

Design the system that turns successful complex tasks into reusable skills.

Include:

- criteria for detecting a candidate skill,
- execution trace capture,
- abstraction process,
- schema generation,
- test generation,
- safety review,
- approval states,
- storage format,
- runtime invocation,
- versioning,
- rollback,
- observability for skill success/failure.

Define what a skill file/spec looks like.
Provide a JSON or YAML schema for a skill definition.
Include examples of at least 3 realistic skills.

8. LLM Provider Abstraction and Routing

Design a provider-agnostic LLM layer.

Support:

- OpenAI,
- Anthropic,
- OpenRouter,
- optional extra providers.

Include:

- unified request/response schema,
- streaming interface,
- structured output interface,
- embeddings abstraction,
- tool-calling abstraction,
- retry logic,
- provider fallback,
- cost accounting,
- latency-aware routing,
- capability-aware routing,
- per-channel model policy,
- per-task policy,
- per-user preference policy.

Provide example adapter interfaces and routing logic.

Explain how to add a new provider in the future with minimal code changes.

9. Tool and Plugin Framework

Design a modular tool system that the agent can use safely.

Include example tools such as:

- web search,
- file read/write,
- local notes access,
- calendar/reminder tools,
- shell-safe command execution,
- HTTP fetch,
- summarization pipelines,
- Telegram/Discord sending,
- email/report generation.

Specify:

- tool registration,
- input validation,
- auth/secrets boundaries,
- permission scopes,
- dry-run mode,
- tool result normalization,
- tool audit logging,
- tool sandboxing for risky actions.

10. CLI and Multi-Channel Interface Design

Design a polished interface strategy.

The CLI should feel first-class, not an afterthought.

Include:

- interactive chat mode,
- command mode,
- slash commands,
- history,
- task status view,
- memory review commands,
- skill management commands,
- profile inspection commands,
- scheduler commands,
- streaming output UX,
- structured logs and rich terminal rendering.

For Telegram and Discord, design:

- message flow,
- auth and account linking,
- chat continuity,
- rate limits,
- command patterns,
- notification formatting,
- attachments handling,
- safe interaction patterns.

11. Scheduling and Automation Engine

Design the automation subsystem in depth.

Include:

- scheduling architecture,
- storage model for jobs,
- recurrence rules,
- cron parsing,
- execution workers,
- retry policy,
- dedupe strategy,
- concurrency policy,
- dead-letter handling,
- alerting,
- audit logs,
- human-in-the-loop approvals,
- failure recovery.

Provide examples for:

- daily summary,
- weekly review,
- scheduled research brief,
- monthly personal goal report,
- recurring cleanup or memory consolidation job.

12. Data Model and Schema Design

Provide concrete data models for the major entities.

At minimum include:

- users,
- identities/accounts,
- sessions,
- messages,
- memories,
- profile facts,
- skills,
- skill versions,
- skill runs,
- tasks,
- tool calls,
- jobs,
- notifications,
- provider configs,
- API keys/secrets metadata,
- audit events.

Show field-level schema suggestions.

Indicate which DB each entity belongs to.

13. Recommended Technology Stack

Recommend a complete stack with rationale.

Cover:

- backend framework,
- orchestration framework,
- primary language,
- CLI framework,
- bot frameworks,
- relational database,
- vector database,
- caching/queue,
- scheduler/worker system,
- observability tooling,
- secret management,
- deployment strategy,
- infrastructure-as-code,
- testing frameworks.

Prioritize:

- low cost,
- strong open-source ecosystem,
- production readiness,
- maintainability,
- solo-builder friendliness.

Include one "recommended default stack" and 2 alternatives:

- ultra-cheap stack,
- more scalable/professional stack.

14. Security, Privacy, and Trust Model

Design security from day one.

Include:

- auth strategy,
- local vs cloud trust boundaries,

- API key handling,
- encrypted secret storage,
- memory privacy controls,
- deletion guarantees,
- backup encryption,
- audit trails,
- least privilege tool access,
- safe shell execution,
- messaging platform risks,
- prompt injection defenses,
- malicious tool output handling,
- rate limiting,
- abuse prevention,
- multi-provider secret isolation.

Explicitly distinguish between:

- data the AI may infer,
- data it may store automatically,
- data requiring explicit consent,
- actions requiring explicit approval.

15. DevOps and Low-Cost Cloud Deployment Plan

Design a realistic always-on deployment model for a cheap cloud server.

Include:

- local development architecture,
- staging vs production,
- Docker strategy,
- reverse proxy,
- process management,
- backups,
- rolling restarts,
- monitoring,
- log retention,
- health checks,
- secret injection,
- domain/TLS,
- failure recovery,
- disaster recovery,
- storage persistence,
- estimated monthly costs.

Assume a solo developer starting on a budget.

Provide at least 3 deployment options with rough trade-offs and monthly cost ranges.

16. API and Module Design

Provide concrete internal module boundaries and external API design.

Include:

- REST or RPC endpoint suggestions,
- webhook handling for Telegram and Discord,
- CLI-to-backend interaction protocol,
- job scheduling API,
- memory management API,
- skill registry API,
- provider management API,
- observability/admin endpoints.

Also provide a suggested monorepo or repo structure.

17. Example Flows and Sequence Diagrams

Provide detailed step-by-step sequence flows for:

1. normal chat request,
2. memory extraction after chat,
3. skill synthesis after a successful repeated task,
4. scheduled daily report execution,
5. Telegram request using the same memory as the CLI,
6. provider failure with fallback routing,
7. user correcting a wrong memory.

Use text-based sequence diagrams.

18. Implementation Roadmap

Provide a phased development plan with realistic milestones.

At minimum include:

- Phase 0: architecture and repo setup,
- Phase 1: core chat + memory MVP,
- Phase 2: CLI + Telegram/Discord gateways,
- Phase 3: scheduling and automation,
- Phase 4: skill generation framework,
- Phase 5: observability, hardening, and scale improvements.

For each phase include:

- goals,
- deliverables,
- dependencies,
- recommended duration,
- critical risks,
- demo criteria,
- "definition of done".

19. Testing and Evaluation Strategy

Design an evaluation framework for the assistant.

Include tests for:

- memory quality,
- retrieval accuracy,
- profile correctness,
- skill usefulness,
- tool safety,
- routing correctness,
- scheduling reliability,
- latency,
- resilience under provider failure,
- regression tracking.

Provide both automated tests and human review loops.

20. Product Risks and Failure Modes

Provide a brutally honest risk analysis.

Include:

- false memories,
- over-personalization,
- privacy leakage,
- brittle skill generation,
- provider API churn,
- cost blowups,
- scheduler misfires,
- bot platform limitations,
- cloud downtime,
- prompt injection,
- destructive automation risks,
- misleading autonomy claims.

For each risk provide:

- description,
- likelihood,
- impact,
- detection method,
- mitigation,
- fallback plan.

21. Final Recommendation

Conclude with:

- your recommended architecture,
- your recommended MVP scope,
- the fastest high-quality path for a solo founder,
- what to build now,
- what to defer,
- what not to build at all initially.

REQUIRED OUTPUT QUALITY BAR

Your answer must be:

- deeply technical,
- structured with clear headings,
- concrete and implementation-oriented,
- explicit about trade-offs,
- opinionated where useful,
- realistic for a solo or small team,
- optimized for long-term extensibility,
- focused on low-cost production viability.

IMPORTANT STYLE RULES

- Do not be generic.
- Do not say "it depends" unless you immediately resolve the trade-off with a recommendation.
- Do not give only options; choose a default architecture.
- Do not hand-wave memory, skills, or scheduling.
- Do not assume a giant team.
- Do not assume enterprise budget.
- Do not hide complexity; surface it clearly.
- Do not confuse "agent" with "just a chatbot wrapper".

- Distinguish what is truly autonomous from what must remain user-approved.
- Make the design robust, original, and practical.

PREFERRED TECHNOLOGY DIRECTION

Bias toward:

- TypeScript or Python, but choose deliberately and justify,
- modular architecture,
- clean interfaces,
- Dockerized deployment,
- Postgres for structured state,
- vector search for retrieval,
- Redis or equivalent for queues/caching where justified,
- low-cost observability,
- safe tool abstractions,
- human-in-the-loop controls.

OPTIONAL STRETCH ITEMS

If useful, also include:

- a file tree for the repository,
- sample environment variable layout,
- starter database schema,
- skill definition examples,
- sample CLI commands,
- event taxonomy,
- job state machine,
- pseudo-code for routing, memory write-back, and skill synthesis.

FINAL INSTRUCTION

Produce the best possible full-system design and roadmap for this assistant as if you were preparing the founding blueprint

Optional wrapper prompt

Use this shorter wrapper before the master prompt if you want more forceful behavior from the model:

I do not want a lightweight answer. I want a deeply reasoned, founder-grade architecture blueprint with production-level

Suggested follow-up prompts

After using the master prompt, you can continue with prompts like:

1. Turn the architecture into a monorepo scaffold

- “Generate the complete folder structure, package list, Docker files, and starter modules for the recommended architecture.”

2. Generate the memory subsystem in detail

- “Implement the memory data model, extraction pipeline, conflict resolution strategy, and retrieval scoring logic in code and schema form.”

3. Generate the LLM provider abstraction layer

- “Write the provider adapter interfaces and starter implementations for OpenAI, Anthropic, and OpenRouter.”

4. Generate the skill system

- “Design and implement the skill registry, skill spec schema, versioning model, and skill execution runtime.”

5. Generate the scheduler

- “Design the recurring task engine, job state machine, retry logic, and daily report workflow.”

6. Generate the CLI and bot gateways

- “Create a polished terminal UX plus Telegram and Discord gateway architecture that share the same backend session and memory.”
-

Notes

- This prompt is intentionally opinionated and long so the model has enough structure to produce an architecture-grade answer rather than a generic brainstorm.
 - If the target model is weaker, ask it to answer section-by-section instead of all at once.
 - For the strongest results, run the output through a second pass asking the model to challenge its own design, identify weak assumptions, and revise the architecture.
-

End state

A good response to this prompt should give you:

- a serious system blueprint,
- a recommended low-cost stack,
- a build order,
- concrete subsystem boundaries,
- storage and deployment decisions,
- a roadmap from MVP to advanced self-improving assistant.